OPSPILOT SYSTEM OVERVIEW & WORKFLOW GUIDE

Complete Architectural Blueprint, User Flows, Role RBAC & Presentation Script

Enterprise AI DevOps & Cloud Operations Copilot | Human-in-the-Loop Architecture

1. Executive Summary & The 30-Second Pitch

> 📌 **The 30-Second Elevator Pitch**
> "OpsPilot is an Enterprise AI Operations Copilot that connects directly to production infrastructure—including GitHub repositories, PostgreSQL databases, and operational productivity channels—using the open Model Context Protocol (FastMCP). It automatically investigates CI/CD failures, analyzes logs, and generates code patches. Crucially, it solves the enterprise safety problem with a Cryptographic Human-in-the-Loop (HITL) boundary: no destructive write operation (like merging PRs or modifying databases) can ever execute without explicit, signed human approval."

2. The Problem Statement: Why OpsPilot Was Built

Modern cloud development and DevOps teams operate under severe operational friction caused by three fundamental challenges:

- **1. Extreme Tool Sprawl & Context Switching:** Engineers must constantly switch between GitHub Actions logs, database consoles (pgAdmin/psql), email alerts, and incident tracking systems just to diagnose a single outage or failed build.
- **2. Slow Mean-Time-to-Resolution (MTTR) & Alert Fatigue:** Developers spend hours manually downloading CI/CD raw terminal logs, sifting through thousands of lines of stack traces, identifying missing dependencies or failing tests, and crafting manual hotfixes under pressure.
- **3. The 'Unsafe Autonomous AI' Paradox:** While modern Large Language Models (LLMs) can generate code, giving an autonomous AI agent direct write access to production Git repos or databases is extremely risky. Traditional AI assistants either have zero execution ability (they just chat) or unchecked execution ability (they can drop tables or break main branches without oversight).

3. The OpsPilot Solution

OpsPilot bridges autonomous AI capability with strict enterprise safety through four pillars:

- **Modular Microservice Orchestration (FastMCP):** Integrates GitHub, PostgreSQL, and Communication channels under a standardized, modular RPC standard (FastMCP). Tools can be added or swapped without touching core agent logic.

- **Intelligent Automated Triage & Root Cause Analysis:** The AI agent automatically parses CI/CD failure logs, detects the root cause (syntax errors, missing packages, DB migration errors), and crafts a concrete diff/patch ready to apply.
- **Human-in-the-Loop (HITL) Governance:** Zero unvetted write actions. Read operations (inspecting logs, listing tables) run seamlessly, while all write operations (creating PRs, executing SQL migrations, sending triage emails) enter a cryptographic approval queue with HMAC-SHA256 signatures.
- **Enterprise Auditability & RBAC:** Every single AI tool call, human approval, rejection, and patch generation is written to an immutable audit ledger for security, compliance, and post-incident reviews.

4. The Two Login Roles: Admin vs. Operator Explained

OpsPilot implements strict Role-Based Access Control (RBAC) to enforce separation of duties between operational investigation and administrative governance:

Feature / Dimension	Lead DevOps Operator (operator123)	Enterprise Administrator (admin123)
Login Credentials	operator@opspilot.internal / operator123 User ID: usr-operator-01	admin@opspilot.internal / admin123 User ID: usr-admin-01
Primary Responsibility	Day-to-day triage, troubleshooting pipelines, debugging DB issues, chatting with AI copilot.	Platform governance, security oversight, approving high-risk changes, compliance auditing.
AI Copilot Capabilities	Full natural language conversation. Can ask AI to analyze logs, inspect schemas, and PROPOSE fixes.	Full conversational access plus administrative overrides and system health diagnostics.
Approval Center Rights	CANNOT approve high-risk actions. Can view status and trigger requests that wait for review.	CAN APPROVE or REJECT pending write operations with cryptographic HMAC signature.
System Management	Read-only access to integrations and workspace members.	Full authority to configure integrations (GitHub/Postgres/SMTP), manage API tokens, and purge logs.

5. End-to-End Workflow: The 5-Step Incident Lifecycle

Here is the exact lifecycle of how a production CI/CD failure is detected, diagnosed, and resolved inside OpsPilot:

- **Step 1: Ingestion & Telemetry Aggregation:** A GitHub Actions build fails (e.g. failing pytest or broken dependency in a repo). GitHub webhooks or periodic pollers notify OpsPilot. OpsPilot syncs the run details, commit metadata, and failure logs into the local operational datastore.
- **Step 2: AI Diagnostic Triage (Operator Interaction):** The DevOps Operator sees the failure on the Pipelines dashboard or asks the AI Copilot: 'What failed in our latest build?'. The AI invokes read-only FastMCP tools (`get_workflow_logs`, `analyze_error_trace`) to parse the log stack trace and isolate the bug.
- **Step 3: Automated Patch Proposal & Security Interception:** The Operator clicks 'Generate AI Fix' or instructs the AI: 'Propose a pull request to fix the dependency error'. The LangGraph agent identifies that `create_pull_request` is a HIGH-RISK WRITE tool. Instead of executing immediately, the system generates a cryptographic Nonce and locks the action in the Approval Center with status PENDING.
- **Step 4: Cryptographic Human Review (Admin Action):** The Enterprise Admin logs in and opens the Approval Center. The Admin inspects the exact tool payload: target repo, branch, commit message, and file diff. If the patch is verified, the Admin clicks 'Approve Request'. OpsPilot generates an HMAC-SHA256 signature containing the timestamp, canonical request hash, and admin ID.
- **Step 5: Autonomous Execution & Immutable Audit Log:** With the cryptographic token verified, the GitHub FastMCP Server executes the pull request creation against the real GitHub repository. The audit log records the entire trace: who requested it, who approved it, the signature hash, and the resulting GitHub PR URL.

6. How Users Interact With the Platform (UI Walkthrough)

The OpsPilot web interface is designed with a sleek, high-contrast dark theme, intuitive status badges, and responsive glassmorphic cards:

- **1. Operations Dashboard:** Displays real-time system metrics: Total Active Incidents, Pending Approvals, Connected MCP Servers, and Mean Time to Resolution (MTTR). Gives an instant pulse of infrastructure health.
- **2. AI Ops Assistant (Chat):** An interactive ChatGPT-style conversational console with streaming responses. Users can ask questions in plain English ('Show me all slow queries on PostgreSQL', 'Why did commit 4f2a fail?'). The AI shows tool call chips as it fetches data.
- **3. CI/CD Pipelines & Failure Triage:** Lists all recent CI/CD pipeline runs across connected repositories. Failed workflows are highlighted in red with badges. Clicking an incident displays the exact terminal error output and a 1-click 'Generate AI Fix' button.
- **4. Approval Center (HITL Gating):** The core safety cockpit. Shows all pending high-risk tool calls awaiting review. Each item displays the requesting user, timestamp, target system, and formatted JSON diff. Admins have 'Approve' and 'Reject' buttons with instant feedback.
- **5. Integrations Center:** Manage connections to external systems: GitHub profile credentials/PAT, PostgreSQL database host & connection strings, and SMTP email relays. Features real-time connection test buttons.
- **6. Audit Ledger:** Provides a searchable, tamper-evident log of every single action performed by both humans and AI agents. Essential for enterprise compliance (SOC2/ISO27001).

7. How to Present This Project (Demo Script & Viva Defense)

> 📌 **Recommended Presentation Flow**
> Follow this exact 4-stage presentation script during your project viva or interview demo. It focuses on business value, architectural elegance, and practical execution.

Stage 1: The Hook & Introduction (1 Minute)

"Good morning/afternoon. Today I am presenting OpsPilot, an Enterprise AI Operations Copilot. In modern cloud environments, when a deployment breaks at 2 AM, engineers are forced to switch between multiple tabs—reading raw GitHub logs, checking PostgreSQL connection pools, and emailing incident teams. OpsPilot automates this entire triage process using AI, but solves the critical safety risk: it guarantees that an AI model can never make destructive changes to production code or databases without human cryptographic authorization."

Stage 2: Explaining the Architecture & MCP (1.5 Minutes)

"Under the hood, OpsPilot is built on Anthropic's Model Context Protocol (FastMCP). Instead of hardcoding API calls into prompts, OpsPilot runs three dedicated microservices: a GitHub MCP server, a PostgreSQL MCP server, and a Productivity MCP server. Our LangGraph AI agent parses engineer intent, selects the right tool dynamically, and fetches live context."

Stage 3: The Live Demo Walkthrough (2.5 Minutes)

- **Step 1 (Dashboard):** "Notice how the dashboard aggregates live metrics across our GitHub repositories and database clusters."
- **Step 2 (AI Assistant):** "I'll ask the AI Copilot: 'Show me any failing workflows in our connected repos'. The AI queries our FastMCP GitHub server and returns the exact failure in seconds."
- **Step 3 (Pipelines Triage):** "We inspect the failed build, view the error trace, and click 'Generate AI Fix'. The AI analyzes the error and drafts a clean patch."
- **Step 4 (HITL Interception):** "Notice what happens when the AI tries to submit the fix: it is BLOCKED by our Human-in-the-Loop boundary. It does not blindly push to GitHub. It enters our Approval Center as a PENDING request."
- **Step 5 (Admin Approval & PR Creation):** "Now, switching to the Admin role, the Admin reviews the exact diff and clicks 'Approve'. An HMAC-SHA256 nonced token verifies the decision, and the GitHub PR is created live. Everything is logged in the Audit Trail."

Stage 4: Defending Common Viva / Interview Questions

- **Q: Why did you use FastMCP instead of standard REST APIs?** A: FastMCP (Model Context Protocol) provides a standardized, model-agnostic contract for exposing data and actions to LLMs. Instead of embedding custom API client code inside prompt templates, FastMCP standardizes tool schemas, input validation, and streaming. If we switch LLMs or add a Kubernetes cluster tomorrow, we only write a new MCP server without touching the AI agent.

- **Q: What prevents the AI from bypassing the approval center?** A: The execution layer requires a cryptographically signed approval token (`nonce.timestamp.hash.signature`) generated by the backend using a server-side secret (`OPSPILOT_APPROVAL_SECRET`). The AI agent does not have access to this secret and cannot generate tokens. Any tool call categorized as 'write' will fail with an HTTP 403 / PermissionDenied unless this valid signature is present.
- **Q: What is the difference between the Admin and Operator logins?** A: The Operator is the 'investigator' who interacts with the AI, inspects logs, and drafts solutions. The Admin is the 'gatekeeper' who manages integrations and cryptographically signs off on destructive actions. This implements the enterprise security principle of Separation of Duties (PoLP).
- **Q: What happens if an external service like GitHub is down?** A: OpsPilot implements a resilient graceful degradation pattern. If the live GitHub API or scraper fails, the MCP server returns cached or structured offline diagnostic information, alerting the user to connectivity issues without crashing the application.